

# CS603 Compiler Design & Parsing Reference Notes

Subject: CS603 | Type: Verified Course Notes | Format: PDF Handbook

## Unit 1: Phases of Compiler & Lexical Analysis

Compiler Phases:

1. Lexical Analysis (Scanner): Converts source character stream into tokens using Regular Expressions & DFAs.
2. Syntax Analysis (Parser): Checks grammatical correctness using Context-Free Grammars and generates a Parse Tree.
3. Semantic Analysis: Type checking and semantic rule verification.
4. Intermediate Code Generation (ICG): Three-Address Code (TAC), Quadruples, Triples.
5. Code Optimization: Machine-independent optimizations (dead code elimination, loop unrolling).
6. Code Generation: Target assembly/machine code emission.

## Unit 2: Top-Down Parsing & LL(1)

Eliminating Left Recursion:

For rule  $A \rightarrow A\alpha \mid \beta$ , replace with:

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$ .

Left Factoring:

For rule  $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ , replace with:

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 \mid \beta_2$ .

FIRST and FOLLOW sets:

- FIRST( $\alpha$ ): Set of terminals that begin strings derived from  $\alpha$ .
- FOLLOW( $A$ ): Set of terminals that can appear immediately to the right of  $A$  in some sentential form.

An LL(1) grammar has no multiple entries in any cell of its parsing table  $M[A, a]$ .

## Unit 3: Bottom-Up Parsing (LR Parsers)

Hierarchy of LR Parsers:

$LR(0) < SLR(1) < LALR(1) < CLR(1)$ .

- LR(0): Items have no lookaheads. Shift and reduce conflicts occur easily.
- SLR(1): Simple LR, places reduce actions only in FOLLOW( $A$ ) of LHS variable.
- CLR(1): Canonical LR with explicit  $[A \rightarrow \alpha \cdot \beta, a]$  lookahead tokens. Most powerful, largest state table.
- LALR(1): Merges CLR(1) states with identical core items to drastically reduce table size.

